Chronic Kidney Disease Prediction – A Machine Learning Based Solution

BIO215 Capstone Project by Group 12

Responsibility	Student Name	Student ID	Contribution
ML model	Yuewei.Xu		40%
Shiny website	Yiming.Huang		30%
R package	Wenbo.Huang		30%

1. Abstract

Chronic kidney disease (CKD) is a major global public health burden. Its early stage is often asymptomatic, which increases the need for reliable prediction tools. Using the Kaggle "Kidney Disease Dataset" (400 patients; 26 features), we built a machine-learning pipeline for binary CKD classification and delivered it as a reproducible software product. We trained three classifiers: random forest, ridge-penalized logistic regression, and support vector machine. All models showed strong performance, with AUROC and AUPRC above 0.90. Support Vector Machine achieved the best test results at a selected decision threshold, with accuracy, AUROC, and AUPRC exceeding 0.99. This level of performance reflects both the model's capacity and the strong diagnostic relationships in clinical measurements to CKD, such as specific gravity and albumin. For reproducibility and usability, the R package encapsulates six core modules: data processing, model training, prediction, model evaluation, model interpretation, and model comparison. The Shiny app extends these functions into three user-facing modules: dataset browsing and visualization, performance comparison with threshold selection, and single-sample plus batch prediction. Overall, the system provides rapid and interpretable predictions, supporting low-resource clinical workflows and offering a practical foundation for future work on CKD screening and prognosis.

2. Introduction

2.1 Public Health Issue

Chronic kidney disease (CKD) refers to abnormalities in kidney structure or function that persist for at least 3 months, commonly reflected by reduced glomerular filtration rate (GFR) and markers of kidney damage [1].

CKD has become a major global public health burden. Population-level evidence shows that CKD contributes substantially to mortality and disability worldwide, with a rising burden across many regions [2]. Patient-facing summaries from the National Kidney Foundation (NKF) also report that CKD affects roughly 10% of the world's population and has climbed in global cause-of-death rankings (reported from 27th to 18th based on earlier Global Burden of Disease analyses) [3]. CKD does not only lead to kidney failure. It also increases cardiovascular risk, which causes mortality. Many CKD patients even die before reaching end-stage kidney disease (ESKD) [4, 5].

Moreover, early detection remains difficult. CKD often has few symptoms until advanced stages, so routine care relies heavily on serum creatinine and estimated GFR (eGFR) [6]. However, serum creatinine varies with muscle mass, diet, sex, age, and other non-renal factors, and it can only rise after a Substantial loss of kidney function (often cited as up to ~50%) [7]. Additionally, these traditional clinical reasoning usually relies on one or a few variables and fixed cut-off thresholds. This linear rule-based approach struggles with multiple clinical risk factors. As a result, many patients in the early, potentially reversible stage are not identified in time and miss the optimal window for intervention.

This combination—high burden, high cardiovascular risk, and weak early clinical signals—creates a clear need for a sensitive early screening approach that integrates multiple routine clinical variables.

2.2 CKD Prediction with Machine Learning

Electronic health records (EHRs) store large volumes of structured clinical data, including demographics, vital signs, laboratory test results, and urinalysis results. These variables can jointly represent kidney health trajectories that are hard to capture with simple linear cutoffs.

Machine learning (ML) can model nonlinear relationships and high-dimensional interactions, which makes it well-suited for clinical risk prediction [8]. Computational nephrology has expanded rapidly, and many methods have been tested for CKD classification. Common choices include support vector

machines (SVM), k-nearest neighbors (KNN), and tree-ensemble models such as Random Forest and XGBoost [9]. However, much of the literature still centers on metric comparisons and leaves two practical gaps [10]:

1. Interpretability gap: Many high-performing models, especially deep neural networks, operate as "black boxes". Clinicians cannot see the basis for a prediction, and evidence-based practice makes it challenging for non-explainable outputs to be incorporated into clinical guidelines.
2. Usability gap: Most studies remain as publications and do not provide software interfaces or web tools that clinicians or patients can use directly. Without such tools, models rarely influence real clinical workflows.

2.3 Data Source and Feature Description

This study utilizes the "Chronic Kidney Disease" dataset from the UCI Machine Learning Repository, which is also hosted on Kaggle [10, 11]. The dataset comprises 400 patient records with 26 columns, collected over a period of nearly two months from a hospital in India.

The class distribution is 250 CKD and 150 non-CKD, which creates a mild imbalance; however, it still supports robust evaluation under careful threshold selection. The 26 columns include one ID column, 24 clinical attributes, plus a binary class label (CKD vs non-CKD). The feature set spans demographics, physical examination, urine findings, and blood biochemistry (Table 1). This breadth aligns with clinical CKD definitions that depend on kidney function measures and kidney damage markers, rather than a single test.

2.4 Machine Learning Techniques

Three classifiers were selected to compare performance and to improve robustness across modeling assumptions: Random Forest provides strong performance on tabular data, resists overfitting, and supports intrinsic importance measures [12]; Ridge logistic regression adds L2 regularization, which stabilizes coefficient estimates under multicollinearity [13]; Support Vector Machine (SVM) captures nonlinear boundaries that often arise in clinical phenotypes [14].

The workflow supports interactive threshold selection using ROC/PR curves to match clinical priorities (e.g., sensitivity-first screening vs specificity-first confirmation). It also promotes a majority-vote strategy to reduce model-specific variance in predictions.

For interpretation, the project uses both feature importance and SHAP values to quantify global and individual-level feature contributions in a consistent framework across models.

2.5 Project Significance

This project addresses key gaps in ML-based CKD prediction by combining high predictive performance with strong interpretability and practical deployment. It delivers an end-to-end data science pipeline, rigorous model development and validation, in-depth explainability analysis, and software translation through an R package and a deployed Shiny web app. This integrated framework offers a practical model for intelligent CKD screening, supporting the broader adoption of precision medicine in nephrology.

ID	Column	Full Name	Type
1	*age*	Age	Numerical
2	*al*	Albumin	Categorical
3	*ane*	Anemia	Categorical
4	*appet*	Appetite	Categorical
5	*ba*	Bacteria	Categorical
6	*bgr*	Blood Glucose Random	Numerical
7	*bp*	Blood Pressure	Numerical
8	*bu*	Blood Urea	Numerical
9	*cad*	Coronary Artery Disease	Categorical
10	*class*	Class (ckd / notckd)	Categorical
11	*dm*	Diabetes Mellitus	Categorical
12	*hemo*	Hemoglobin	Numerical
13	*htn*	Hypertension	Categorical
14	*id*	ID	Numerical
15	*pc*	Pus Cell	Categorical
16	*pcc*	Pus Cell Clumps	Categorical
17	*pcv*	Packed Cell Volume	Numerical
18	*pe*	Pedal Edema	Categorical
19	*pot*	Potassium	Numerical
20	*rc*	Red Blood Cell Count	Numerical
21	*rbc*	Red Blood Cells	Categorical
22	*sc*	Serum Creatinine	Numerical
23	*sg*	Specific Gravity	Categorical
24	*sod*	Sodium	Numerical
25	*su*	Sugar	Categorical
26	*wc*	White Blood Cell Count	Numerical

Table 1. *Feature dictionary and measurement notes for the "Chronic Kidney Disease" dataset.*

3. Materials and Methods

This project implemented an end-to-end machine learning workflow, covering data cleaning, model training, class prediction, performance evaluation, and model interpretation. To maximize reproducibility and reuse, we packaged the full pipeline as an R package and deployed an interactive Shiny application for result dissemination (Figure 1).

3.1 Data Pre-processing

The raw clinical table contains missing values and format artefacts; using it directly would introduce unstable parsing and biased estimates. We therefore applied a strict pre-processing pipeline (Figure 1), implemented as the function *data_process_ckd()*. This function returns:

1. a row-level missingness summary (see Section 3.1.2);
2. a column-level missingness summary (see Section 3.1.4);
3. the processed training and testing datasets (see Sections 3.1.3–3.1.4).

3.1.1 Data Cleaning

During CSV ingestion, non-standard tokens were detected in multiple fields, including tab characters, stray blanks, and placeholder symbols (e.g., \t, whitespace, ?). All such artefacts were recoded as NA, after which manual screening confirmed that each entry represented either a valid measurement, category, or a missing value.

All variables were then assigned to either a numerical or categorical type. Numerical features (Table 1) were each explicitly converted into numeric to prevent silent type inconsistencies. Categorical variables (all binary) (Table 1) were encoded using a consistent reference scheme: the clinically negative class was mapped to 0, and the alternative class to 1, thereby improving standardization and simplifying downstream modeling.

3.1.2 Row-level Missingness Profiling

After format standardization, missing entries (NA) remained and were addressed first at the row level to remove patients with severe incompleteness. For each record (patient), the fraction of missing features was computed as the number of NA features divided by the total number of features. Records with more than 70% missing features were labeled as “dirty rows” and removed, yielding 369 usable records for downstream analysis. This cutoff prioritizes data validity while preserving samples under a limited cohort size. This threshold is not fixed; the parameter row_threshold in the function *data_process_ckd()* allows the missingness threshold to be adjusted as needed.

3.1.3 Train–Test Dataset Split

To minimize the risk of information leakage during manipulations, we split the dataset before filling in the missing values. We randomly assigned 70% of the records to the training set and the remaining 30% to the testing set. Random sampling helps maintain a similar class composition (*ckd* vs. *notckd*) across splits. The split ratio is configurable through the train_percentage parameter in the *data_process_ckd()* function.

3.1.4 Column-level Missingness Imputation

Missing values were processed separately in the training and testing datasets using a tiered strategy. For each feature, the missingness rate was summarized as the proportion of NA entries among all patients. Features were then assigned to three levels: low (<5%), moderate (≥5% and <20%), and high (≥20%) missingness.

Imputation in the training set followed the missingness tier and the measurement scale of each feature.

1. Low missingness (<5%): simple univariate imputation was applied using median or mode. The mean was avoided to reduce sensitivity to outliers. The choice between median and mode was determined through a manual review, with reference to the clinical and biological interpretation of each feature. Determined imputation methods are listed below: median imputation: *age, bp, bu, sc*; mode imputation: *ane, appet, ba, cad, dm, htn, pcc, pe.*
2. Moderate missingness (≥5% and <20%): to retain class-dependent distributions during model fitting, class-stratified median/mode values were computed separately within *ckd* and *notckd*, and each record was imputed using the statistic corresponding to its class label. This design reduces the risk that global imputation obscures subgroup-specific patterns that may support discrimination. Determined imputation methods are listed below: median imputation: *al, bgr, hemo, pcv, pot, sg, sod, su*; mode imputation: *pc.*
3. High missingness (≥20%): high missingness was treated as potentially informative because measurement availability itself may reflect clinical decision-making.

Therefore, class-stratified mode imputation was paired with an explicit missingness indicator for each high-missingness feature, coded as 0 (missing in the raw record) and 1 (observed), to preserve this signal and support interpretability. Determined imputation methods are listed below: mode imputation: *rbc, rc, wc*; missingness indicators added: *rbc_missing, rc_missing, wc_missing*.

Testing-set imputation was designed to prevent information leakage. It did not use class-stratified imputation to avoid exposure to the ground-truth label (*class*) before model training. Consequently, all testing-set features received non-stratified median/mode imputation; however, high-missingness indicators were still added to retain information conveyed by measurement availability. Determined imputation methods are listed below: median imputation: *age, al, bgr, bp, bu, hemo, pcv, pot, sc, sg, sod, su*; mode imputation: *ane, appet, ba, cad, dm, htn, pc, pcc, pe, rbc, rc, wc*; missingness indicators added: *rbc_missing, rc_missing, wc_missing*.

3.2 Model Training

Model training was implemented through a single function, *model_train_ckd()*. The argument *type* selects the learning algorithm: *1* – Random Forest (RF), 2 – Ridge Logistic Regression (GLM), and 3 – Support Vector Machine (SVM). In all cases, the model consumes the complete set of clinical predictors (the 24 original variables plus three missingness indicators introduced during pre-processing) and targets the binary outcome classification (*ckd* vs. *notckd*).

Default configurations were chosen to provide stable baselines while preserving extensibility. The random forest model uses package defaults without additional tuning. The ridge logistic regression is fit via a binomial GLM (*family = "binomial"*) with an L2 penalty (*alpha = 0*) and feature standardization enabled (*standardize = TRUE*) so that regularization acts comparably across predictors with different scales; this setting typically shrinks coefficients without forcing zeros, which suits clinically meaningful covariates and supports feature importance calculations. The support vector machine employs a radial basis function kernel (*kernel = "radial"*) and enables probabilistic outputs (*probability = TRUE*) to support binary classification.

Algorithm-specific parameters for *randomForest::randomForest()*, *glmnet::cv.glmnet()*, and *e1071::svm()* remain user-configurable in our function. The function returns a compact list containing a model label and the fitted model object for downstream prediction and evaluation.

3.3 Prediction

Two prediction modes are provided. Probability estimation uses the full feature set in the input table and generates record-level CKD diagnosis via *predict_ckd()*, which internally calls *stats::predict()* for the selected learner. A second interface, *predict_class_ckd()*, converts these probabilities into discrete labels by applying a user-specified threshold (default 0.5). As an example, under *threshold = 0.5*, records with predicted CKD probability ≤ 0.5 are assigned *notckd*, whereas probabilities > 0.5 are assigned *ckd*.

3.4 Model Evaluation and Comparison

Model assessment was designed to support both single-model evaluation and cross-model comparison.

Single-model performance is computed by *model_evaluation_ckd()*. The function returns a structured output comprising three components: *metrics*, *curves*, and *best_threshold*:

1. The *metrics* component reports the confusion matrix counts and derives standard classification statistics: Accuracy, Precision, Recall, and False Positive Rate (FPR)
2. The *curves* component contains both numeric summaries and ggplot2 objects for the Receiver Operating Characteristic curve (ROC, Recall vs FPR) and the Precision–Recall Curve (PRC, Precision vs. Recall).
3. The *best_threshold* component provides a recommended operating threshold selected by maximizing the F1-Score, together with the corresponding performance metrics under that threshold.

For between-model comparison, *model_comparison_ckd()* evaluates multiple learners under a standard evaluation protocol. The function accepts a list of fitted model objects, each in the list format returned by *model_train_ckd()*. Thresholding can be specified either as a single shared value (*threshold*) or as a model-specific vector (*threshold_list*), aligned to the order of *model_list*. When both are provided, *threshold_list* takes precedence. For each model, the function computes Accuracy, Precision, Recall, Specificity, F1-Score, AUROC, and AUPRC at the corresponding thresholds, and summarizes results in a consolidated bar plot for visual comparison.

3.5 Model Interpretation

Model interpretation was implemented for all three classifiers through two complementary layers: model-specific feature importance and SHAP-based attribution.

Feature importance is generated by *model_interpretation_importance_ckd()*. Importance scores are defined in a manner consistent with each algorithm's inductive bias (see Section 4.2.1).

To provide attribution at both dataset and instance resolution, SHAP explanations are generated using the *model_interpretation_shap_ckd()* function. Three SHAP views are available for every model.

1. Global SHAP summarizes the contribution of all features across the dataset (*visualization_type = 1*);
2. Single-feature SHAP isolates the effect of a specified predictor on predictions across samples (*visualization_type = 2*);
3. Single-row SHAP decomposes an individual prediction into feature-wise contributions for a selected record (*visualization_type = 3*).

3.6 Products and Version

This study delivers reusable research artefacts through an open-source software package and an accompanying interactive web application.

The full R package is publicly available on GitHub [15] and consolidates the complete workflow into eight callable functions divided into six modules:

1. *data_process_ckd()*
2. *model_train_ckd()*
3. *predict_ckd()* & *predict_class_ckd()*
4. *model_evaluation_ckd()*
5. *model_comparison_ckd()*
6. *model_interpretation_importance_ckd()* & *model_interpretation_shap_ckd()*

A comprehensive README.md document further details installation, provides minimal working examples, and outlines core usage patterns to support rapid adoption and independent verification.

To facilitate accessible evaluation and model deployment, an online Shiny application is deployed [16]. The interface organizes functionality into three coherent modules:

1. dataset browsing and visual analytics for exploratory inspection;
2. side-by-side model performance comparison with explicit decision-threshold selection;
3. prediction utilities supporting both batch inference and single-record prediction, enabling direct translation from trained models to practical screening scenarios.

The computational environment and package versions used for all experiments are reported below. For R package: Core runtime: R (v4.4.3); Base utilities: stats (v4.4.3), utils (v4.4.3); Modelling engines: randomForest (v4.7.1.2), glmnet (v4.1.10), e1071 (v1.7.16); Performance evaluation: precrec (v0.14.5); Interpretability: shapviz (v0.10.3), fastshap (v0.1.1); Visualisation: ggplot2 (v4.0.1). For Shiny web application: Core runtime: R (v4.4.3); Web framework: shiny (v1.12.1), shinydashboard (v0.7.3), shinyjs (v2.1.0), DT (v0.34.0), ggplot2 (v4.0.1); Analytical backend: ckdPrediction (v1.0.0).

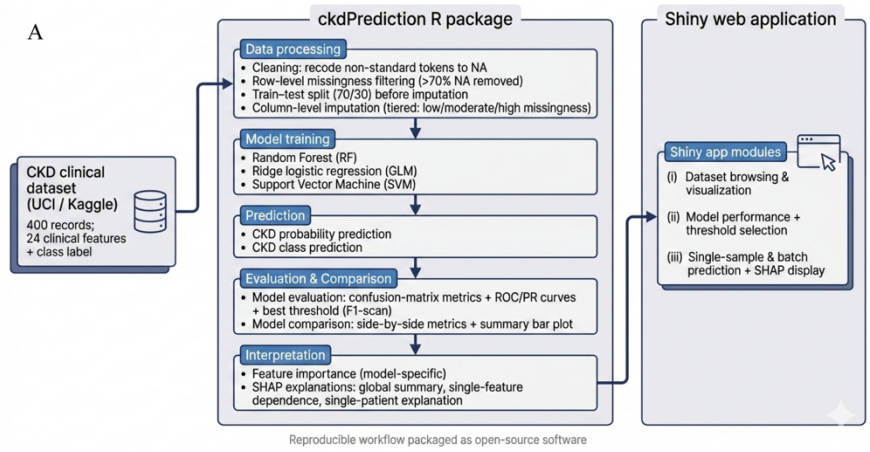


Figure 1. (A) *End-to-end project workflow and software delivery generated by Gemini 3. Data flows from the raw clinical table through preprocessing, model training, evaluation, and interpretation. Results are encapsulated as an R package and a Shiny web application.*

4. Results

All results reported in this section were generated under a fixed random seed in R (*set.seed(123)*) to ensure reproducibility.

4.1 Performance Evaluation

4.1.1 Model Usability

After preprocessing, the training data retained a mild class imbalance (*ckd*:*notckd* = 219:150). Model usability was therefore assessed using threshold-scanned evaluation curves that remain informative under imbalance. Specifically, the decision threshold on the predicted positive-class probability was varied, and the corresponding metric values were computed at each operating point to form systematic performance profiles (Figure 2 A, B, C). Area-under-curve (AUC) statistics summarized overall usability. As reported in Table 2, AUROC values for random forest (RF), ridge logistic regression (GLM), and support vector machine (SVM) were 0.997, 0.998, and 1.000, respectively (all > 0.8), while AUPRC values were 0.998, 0.999, and 1.000 (all > 0.7). Collectively, these results indicate consistently strong discriminative performance across models despite the mild class imbalance.

4.1.2 Threshold Recommendation

AUC metrics demonstrate discriminative ability, yet model employment still requires an explicit decision threshold for converting predicted probabilities into binary labels.

Threshold choices determine the points on ROC and PRC curves and therefore control the trade-off between missed cases and false alarms. At any fixed threshold, predictions fall into four categories: true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). As the threshold increases, fewer samples are labeled as positive; consequently, TP typically decreases, while FP typically decreases, whereas FN usually increases, and TN usually increases. This monotonic shift explains the geometry of the evaluation curves. On the ROC curve, each threshold corresponds to a ratio:True Positive Rate (TPR) / False Positive Rate (FPR). Moving toward stricter (higher) thresholds generally lowers both TPR and FPR, reflecting a more conservative rule that reduces false positives at the cost of increased misses. On the PRC curve, thresholding induces a trade-off between precision and recall. Precision often increases when FP declines faster than TP, but it may stagnate or fall if TP collapses more rapidly; thus, the PR curve makes explicit the balance between positive-call reliability and case coverage. These basic relationships enable the initial selection of a qualitative threshold by inspecting the curves.

A quantitative recommendation was then derived using the $F1 - Score = (2 * Precision * Recall)/(Precision + Recall)$. By evaluating F1 across thresholds, a model-specific optimal threshold was identified for each classifier (Figure 2 D; Table 3) and reported as the recommended operating cutoff. In summary, our recommended thresholds for models achieve a relatively high score for both precision and recall in the PRC curve. The corresponding ROC locations also exhibit comparatively low FPR, supporting discrimination between true negatives and false positives under mild imbalance. Nevertheless, threshold selection remains application-dependent. Because class imbalance can make ROC-based summaries appear overestimated, PRC-based considerations are recommended for prioritizing. In addition, practical costs should inform the final cutoff. For example, if the cost of verifying a false negative sample is much lower than excluding a false positive sample, then we could enhance the cutoff to decrease the expense while maintaining effectiveness.

4.1.3 Model Metrics and Comparisons

Using the model-specific thresholds derived in Section 4.1.2, point-estimate metrics were calculated for each learner (Table 3) and summarized for cross-model comparison (Figure 2 E). Across models, discrimination on the testing set remained consistently strong, with accuracy ranging from 0.982 to 1.000, AUROC from 0.997 to 1.000, and AUPRC from 0.998 to 1.000, indicating near-ceiling performance under the recommended operating points. Notably, the SVM achieved a score of 1.000 across all reported metrics on this testing split, consistent with perfectly separated predictions.

Recall-focused performance was uniformly high. Under the recommended thresholds, RF, GLM, and SVM all achieved a Recall (TPR) of 1.000, implying an FN of 0 and thus no missed CKD cases in the testing set. In contrast, specificity distinguished the models: RF and GLM achieved Specificity = 0.956 (equivalently, FPR = 0.044), reflecting a small number of non-CKD samples misclassified as CKD (FP > 0), whereas SVM reached Specificity = 1.000 with FP = 0.

As a result, SVM appears to be our best model, with the highest performance. However, the apparent “perfect” result warrants stricter validation to confirm generalizability beyond the current split.

Model	AUCROC	AUCPRC
RF	0.997	0.998
GLM	0.998	0.999
SVM	1.000	1.000

Table 2. Discrimination performance summary on the testing set, reporting AUROC and AUPRC for random forest (RF), ridge logistic regression (GLM), and support vector machine (SVM).

Model	Threshold	F1-Score	FPR	Recall (TPR)	Precision	Accuracy
RF	0.493	0.985	0.044	1.000	0.971	0.982
GLM	0.297	0.985	0.044	1.000	0.971	0.982
SVM	0.541	1.000	0.000	1.00	1.000	1

Table 3. Model-specific point-estimate metrics at the recommended thresholds. Models include random forest (RF), ridge logistic regression (GLM), and support vector machine (SVM).

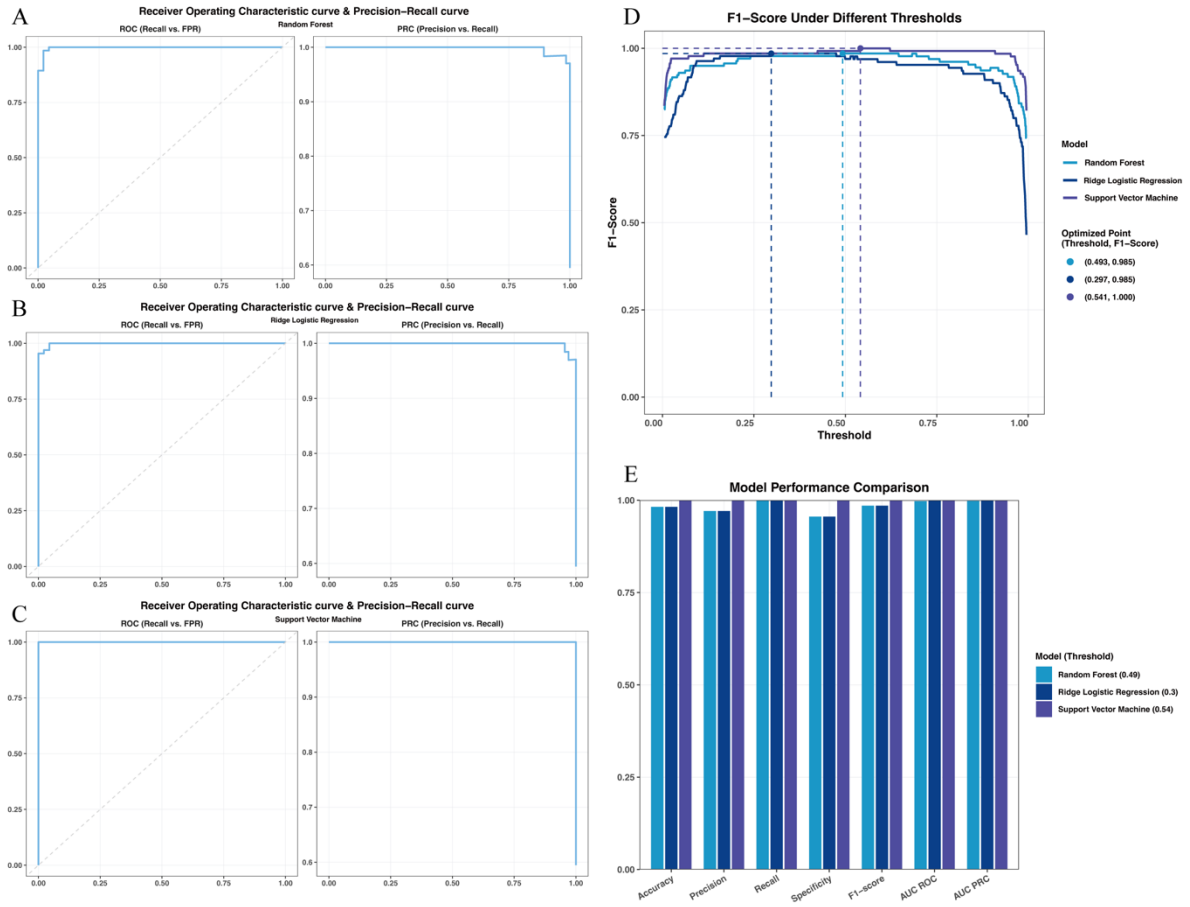


Figure 2. ***(A)*** *ROC & PRC curves for Random Forest.* ***(B)*** *ROC & PRC curves for Ridge Logistic Regression.* ***(C)*** *ROC & PRC curves for Support Vector Machine.* ***(D)*** *F1-Score scanning across all thresholds for three models, with the selected optimum marked.* ***(E)*** *Cross-model performance comparison at the recommended operating thresholds, summarizing point metrics.*

4.2 Model Interpretation

4.2.1 Feature Importance

Feature importance was first used to characterize the reasoning behind models. As RF, GLM, and SVM rely on different learning principles, importance was quantified using model-specific definitions and is therefore not directly comparable in scale across models; nevertheless, ranking patterns remain informative.

1. For random forests (Figure 3 A), importance is quantified by $MeanDecreaseGini$ aggregating the reduction in Gini impurity attributable to splits on each predictor across all trees; larger values indicate a greater contribution to node purity.
2. For ridge logistic regression (Figure 3 B), importance is quantified by $|\beta_i| \times sd(x_i)$, which rescales the absolute coefficient magnitude by the predictor's variability to yield a comparable influence on the log-odds scale.
3. For support vector machines (Figure 3 C), importance is computed via $Permutation\ \Delta Accuracy$. Each feature is permuted across samples, and the resulting decrease in classification accuracy is recorded, with larger drops indicating more substantial reliance of the decision function on that feature.

A shared signal emerged across models. Features repeatedly ranked near the top concentrated in renal function and urinalysis, together with anaemia-related markers, notably Specific Gravity, Albumin, and Hemoglobin.

Model-specific preferences also appeared: RF emphasized haematology and urinalysis, with particularly strong contributions from Packed Cell Volume, Red Blood Cell Count, Hemoglobin, and Specific Gravity. GLM placed Specific Gravity as the dominant predictor and ranked Red Blood Cells (Missing) unusually high, highlighting that the signal aligns with the missingness structure. SVM prioritized Albumin, then Specific Gravity, Hemoglobin, illustrating a pattern spanning urinalysis and inflammation.

Notably, missingness indicators ranked highly in GLM and SVM. Such prominence often indicates non-random missingness, plausibly reflecting clinical test ordering and documentation processes that correlate with CKD status.

Because importance rankings do not encode effect direction, a complementary analysis based on SHAP was conducted below to quantify directional contributions at both global and record levels.

4.2.2 SHAP – Summary-level Explanation

SHAP (SHapley Additive exPlanations) was applied to all three models to obtain directional, additive attributions of feature contributions (Figure 3 D, E, F).

Across learners, the SHAP summary plots converged on a consistent risk profile. Low Specific Gravity produced concentrated, strongly positive SHAP values, indicating a stable shift of predictions toward CKD, whereas higher Specific Gravity tended to contribute negatively. Anaemia-related variables showed coherent directionality: lower Hemoglobin, lower Packed Cell Volume, and lower Red Blood Cell Count predominantly appeared with positive SHAP values, implying systematic increases in predicted CKD probability when these markers declined; higher values more often aligned with negative contributions. Complication features also contributed: the "present" states of Hypertension and Diabetes Mellitus were enriched among positive SHAP contributions, consistent with an increased predicted risk.

Model-specific attribution patterns complemented the shared signal. RF exhibited broader SHAP spread for haematology and urine concentration features, consistent with heavier reliance on Specific Gravity and anaemia-related markers. GLM placed greater emphasis on the missingness structure: Red Blood Cells (Missing) ranked highly, and the "missing" state was associated with more pronounced positive SHAP values, suggesting that the missingness pattern itself carries discriminative information in this dataset. SVM showed consistency with RF, while assigning clearer positive attribution to complications (Hypertension/Diabetes Mellitus).

4.2.3 SHAP – Case-level Explanation

Beyond global summaries, case-level SHAP explanations were generated for each model to support feature-specific and record-specific interpretation (e.g., Figure 4 A, B).

Single-feature dependence views illustrate how variation in an individual feature translates into changes in SHAP contribution.

In contrast, patient-level explanations decompose each predicted CKD probability into feature-wise additive effects. This instance-based decomposition provides a transparent rationale for individual classifications and can assist clinical review by linking a patient's measured profile to the model's decision logic.

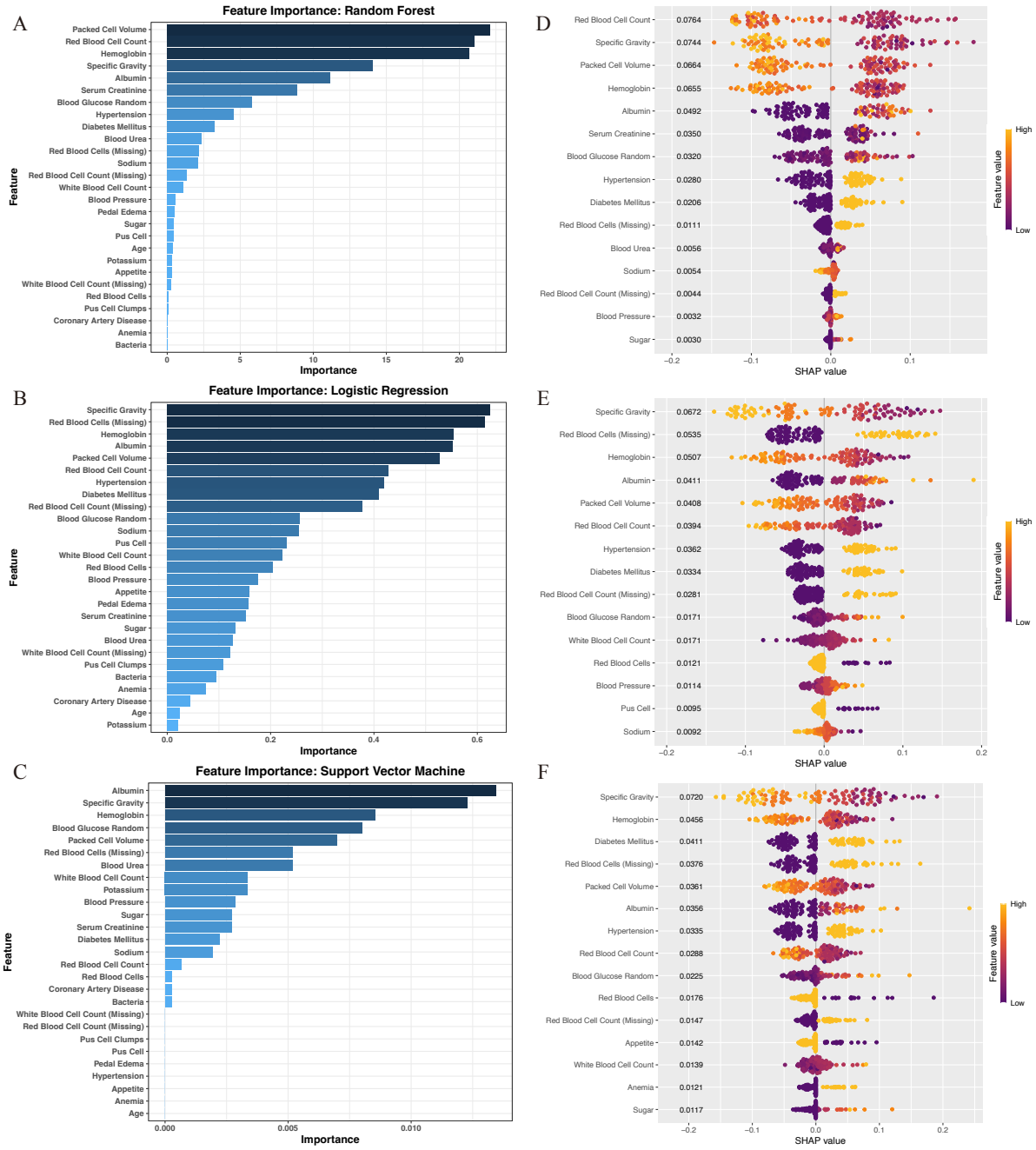


Figure 3. *Model-specific interpretation visualizations.* ***(A)*** *Feature importance for random forest.* ***(B)*** *Feature importance for ridge logistic regression.* ***(C)*** *Feature importance for the support vector machine.* ***(D)*** *Global SHAP for random forest.* ***(E)*** *Global SHAP for ridge logistic regression.* ***(F)*** *Global SHAP for support vector machine.*

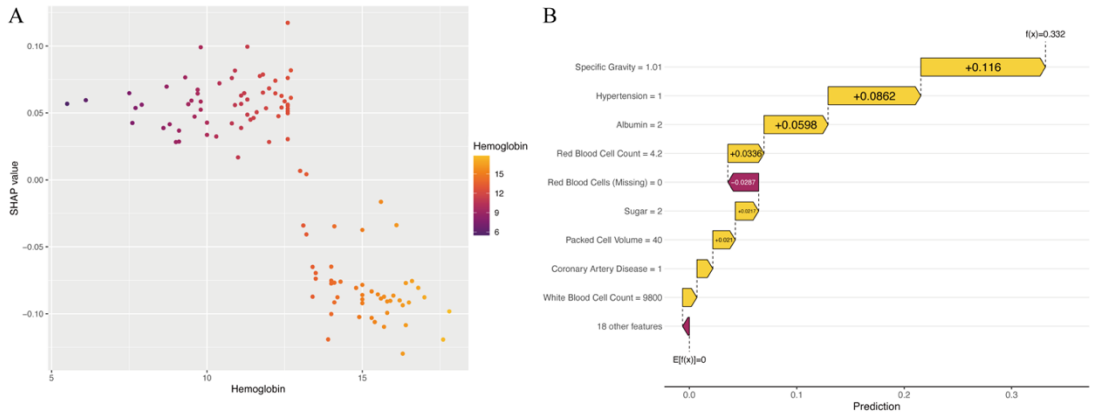


Figure 4. *Case-level SHAP interpretation for a support vector machine.* ***(A)*** *Feature-level SHAP for Hemoglobin (hemo).* ***(B)*** *Record level SHAP for a random patient.*

5. Web Usage

To translate the trained machine-learning models into an accessible, reader-friendly tool, a Shiny-based web application was developed and deployed at https://yueweixu.shinyapps.io/ckdPrediction [16]. This section serves as a practical guide for navigating the interface and running the main functions.

5.1 Interface overview

After opening the webpage, a dashboard layout appears. The left sidebar provides navigation to four core modules: Home Page, CKD Data, Model Performance, and CKD Prediction. A "Reinitialize app" button allows refreshing of the webpage and redirects to the Home Page.

5.2 Home Page

The Home Page provides a brief overview of the project and a concise introduction to its usage. A complete workflow diagram (Figure 5 A) is displayed to summarize how data moves through the system. A feature dictionary is also included for quick reference when interpreting variable names.

5.3 CKD Data

The CKD Data module offers an interactive view of the kidney disease dataset and basic visual exploration. It contains two subpages.

5.3.1 Dataset Browser

This page displays the raw dataset in an interactive table. It supports three types of table operations:

1. Apply the pipeline processing to the dataset (implicitly using *data_process_ckd()* function in the R package);
2. exclude rows with missing values (NA) to obtain a complete-case view;
3. filter variables in detail, using sliders for numerical features and checkboxes for categorical features.

After customization, the updated table is shown. The table can be sorted by any feature and searched by keyword. Download options are provided so that either the original dataset or the customized dataset can be exported for offline analysis.

5.3.2 Data Visualization

This subpage utilizes the customized table from the Dataset Browser (Section 5.3.1) as input and generates plots for each feature. Numerical features are shown as histograms stratified by CKD label (*ckd* vs. *notckd*), and categorical features are shown as stacked bar plots, reporting the proportion of *ckd* vs. *notckd* within each category. Each figure can be downloaded in both PNG and PDF formats. (Figure 6 A)

5.4 Model Performance

The Model Performance module supports threshold selection, model comparison, and feature-importance inspection. It contains three subpages.

5.4.1 Threshold Selection

This page presents ROC and PR curves for each of the three models. A candidate threshold can be chosen in two ways:

1. clicking a point on the curve;
2. Entering a numeric value in the threshold input box.

Once a threshold is selected, the page displays the corresponding metrics at that point to support decision-making. After choosing a threshold, the selected point on both curves and the numeric input box remain synchronized. All plots provide PNG and PDF download options.

Because the curves are displayed with plotting simplifications, the curve geometry may not perfectly match the underlying ROC/PR curves. This page is intended primarily for threshold selection. For exact AUC values, refer to the metrics shown on the next page or use *model_evaluation_ckd()* in the R package.

5.4.2 Model Comparison

This page allows model-specific thresholds to be specified (defaults are set to the recommended thresholds reported in Section 4.1.2). After clicking Evaluate & Compare, the system displays (Figure 6 B):

1. each model's performance metrics;
2. a consolidated comparison plot. Results and figures can be downloaded in PNG and PDF formats.

5.4.3 Feature Importance

This page displays feature-importance plots for RF, GLM, and SVM, together with short notes describing the corresponding algorithms. Each plot can be exported as PNG or PDF.

5.5 CKD Prediction

The CKD Prediction module provides two prediction modes: batch prediction for multiple patients and single-patient prediction. It contains two subpages.

5.5.1 Batch Prediction

Batch Prediction supports uploading a CSV file containing patient measurements and generating predictions in bulk. Users can first upload data in CSV format (the input file should be processed using *the data_process_ckd()* function). Alternatively, an example dataset can be loaded directly from the app. The input table is displayed for confirmation before execution. Then, thresholds are allowed for modification for each model (defaults follow the recommended values in Section 4.1.2). After clicking "Run Prediction", six new columns are appended to the dataset (two columns per model: predicted class label and predicted CKD probability). Finally, a SHAP Interpretation panel is provided. Users select a target model and an interpretation mode (global summary, single-feature view, or single-patient view), then click Run SHAP to generate the corresponding plots (SHAP computation is relatively expensive; plot generation may take several seconds). All outputs support PNG and PDF downloads.

5.5.2 Single-patient Prediction

Single-patient Prediction enables real-time prediction from manually entered measurements. Users can first set thresholds for each model (defaults are outlined in Section 4.1.2) and enter patient measurements feature by feature, or load a provided random example patient. Then, predictions are updated immediately, which report a final CKD decision based on majority voting across the three models, each model's individual class vote, and each model's predicted CKD probability. A SHAP Interpretation panel is also available for patient-specific explanation. After selecting a model and clicking Run SHAP, the app generates a patient-level attribution plot (this step may take a few seconds). All figures can then be downloaded in PNG and PDF formats. (Figure 6 C)

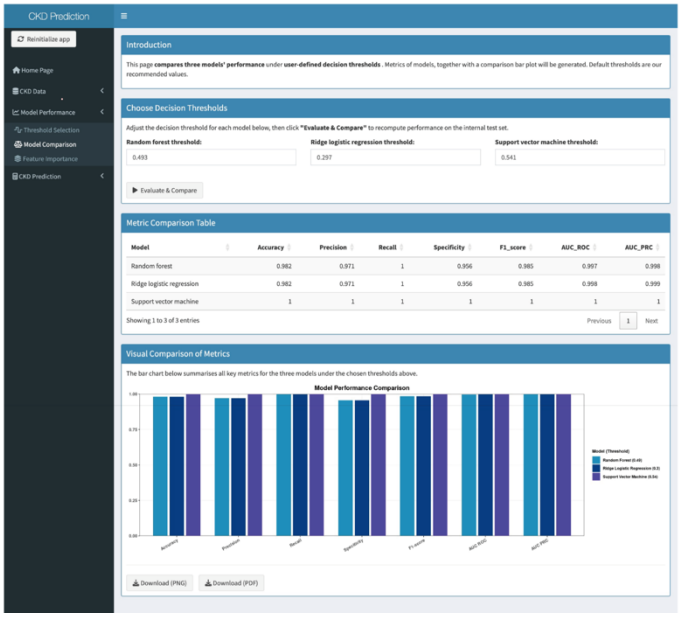


Figure 5. (A) *Web-application workflow diagram generated by Genimi 3.*

A



B



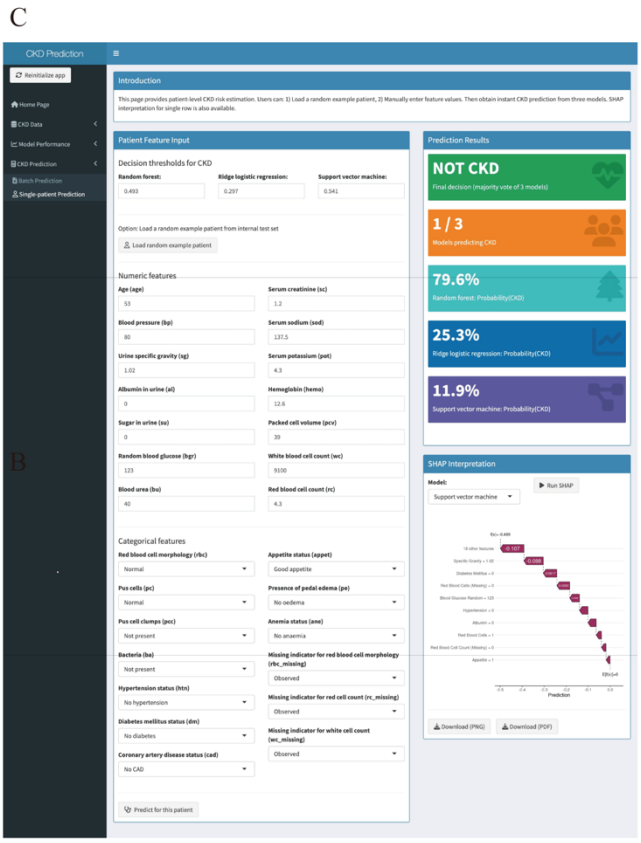


Figure 6. *Screenshot demonstrating some checking results of the Shiny web application.* ***(A)*** *CKD Data – Data Visualization: distribution patterns of features across different CKD labels.* ***(B)*** *Model Performance – Model Comparison: model metrics and comparison bar chart under specified thresholds.* ***(C)*** *CKD Prediction – Single-patient Prediction: prediction of a random patient, with record-level SHAP plot generated.*

6. Discussion and Conclusion

6.1 Biological Relevance of the Dataset

This study involves a clinically collected CKD dataset acquired from a hospital in India over two months. The feature set encompasses both numerical and categorical measurements, providing a comprehensive snapshot of patient-level data. Crucially, because the data originate from routine hospital assessment rather than experimental profiling, the task is best framed as risk stratification and diagnostic support based on standard clinical testing, rather than as mechanistic inference at the molecular-omics level; this positioning strengthens its real-world utility.

Several observations from exploratory analyses and model interpretation deepen the understanding of both CKD and the dataset. First, the distributions across variables indicate that confirmed cases of CKD cluster predominantly in middle-aged and older individuals, reinforcing the importance of prioritizing screening and prevention in these age groups. Second, feature-importance patterns across all three models consistently emphasize urinalysis and renal-function–related measures, together with anaemia-associated indicators (e.g., Specific Gravity, Albumin, and Hemoglobin). This concordance aligns with established CKD phenotypes, including impaired urine concentrating ability, proteinuria as a marker of renal injury, and CKD-associated anaemia. It also supports the feasibility of using these routinely available measurements as low-cost candidate biomarkers for the efficient detection and prognosis of CKD. Third, SHAP-based explanations suggest substantial co-occurrence signals between CKD and common comorbidities such as hypertension and diabetes mellitus, underscoring the clinical burden of complications and motivating earlier CKD screening among patients with these conditions. Finally, the inclusion of missingness indicators revealed that missingness itself carried considerable explanatory weight, implying that data absence may be informative rather than random and may

reflect clinical ordering or documentation practices (for instance, suspected CKD cases receiving more comprehensive testing). This phenomenon is practically relevant for health-informatics modelling, yet it may also reduce cross-site transportability because workflows and recording conventions may vary between hospitals.

6.2 Model Performance

Across all three models: random forest (RF), ridge logistic regression (GLM), and support vector machine (SVM), the predictive signal proved strong and consistent under mild class imbalance (AUROC 0.997–1.000; AUPRC 0.998–1.000). To translate predicted possibilities into classification, we selected model-specific operating points using an F1-score scanning for each model.

Under these thresholds, overall performance converged across models and approached the empirical upper bound (Accuracy 0.982–1.000), indicating effective separation of CKD from non-CKD on the test set.

From a clinical perspective, all three models achieved perfect sensitivity (Recall/TPR = 1.000), eliminating false negatives and aligning with a "reduce-missing" clinical priority. The practical differences, therefore, concentrated on false-positive control: RF and GLM showed identical specificity (0.956; FPR = 0.044), whereas SVM reached a specificity of 1.000, implying no misclassification of non-CKD as CKD in the test set and, consequently, a more conservative positive prediction.

In parallel, interpretability analyses strengthened clinical plausibility: each model provided feature-importance profiles and SHAP-based local explanations, enabling attribution of predictions to both variables and their values, which directly supports transparent clinical reasoning.

Overall, the results indicate high confidence, high accuracy, and explanation-ready predictions, with SVM offering the best performance.

6.3 Biological and Health Utility

This framework supports low-cost, routine-test-driven CKD risk screening and examination guidance. The input features are derived from standard clinical assessments, making the models suitable for practical use with adjustable decision thresholds. The model explanations further strengthen practical value at the point of interpretation. In addition, the released R package and the deployed Shiny web application convert this project from a one-off analysis into a reusable research asset, enabling others to reproduce the workflow, evaluate the models, and extend the system for subsequent studies related to CKD.

6.4 Limitations and Future Directions

1. Dataset-level limitations: the dataset originates from a single hospital and covers a short acquisition window (approximately two months). Consequently, generalizable and geographic demographic problems exist. Sample representativeness, testing process, and the underlying disease pattern may differ across regions, hospitals, and ethnic groups. Future work should therefore expand sampling across multiple hospitals and geographically diverse populations to provide broader training and testing evidence.
2. Model-level limitations: the test set remains relatively small and mildly imbalanced; although computational strategies can mitigate some consequences, these constraints still weaken the reliability of point-estimate performance. In addition, the evaluated models, particularly the SVM, achieved extremely high-scoring metrics. We attribute this partially to the presence of several routine clinical variables that are closely linked to CKD diagnosis; such features can yield strong separation or implicit causality for CKD and are suitable for diagnostic support. The underlying clinical mechanisms and the extent to which the model depends on particular inputs, therefore, warrant further investigation. Methodologically, nested cross-validation would provide a more rigorous assessment of stability, clarify the relationship between input features and performance, and reduce the risk of overestimating.
3. Deployment-level limitations: the model treats missingness indicators as influential signals, implying that the absence of measurements may reflect clinically meaningful processes rather than random omission. Therefore, a more standardized diagnostic workflow is desirable to specify which measurements should be obtained and which additional tests are indicated at different CKD stages, both to support harmonized clinical practice and to enable deeper, more reliable model interpretation.

6.5 Conclusion

Overall, this study developed and operationalized a clinically useful machine-learning framework with high predictive credibility, strong performance, and transparent interpretability for binary CKD prediction using clinical measurements, and delivered it as reproducible software (an R package and a Shiny application). The analyses consistently highlighted urinalysis/renal-function and anaemia-related indicators as core contributors to CKD discrimination, supporting the potential of this framework as a low-cost screening tool. Nevertheless, considering that conclusions remain constrained by the limited sample size and by the imperfect resolution of model interpretation, future work is still necessary and requires effort.

7. Appendices (Project URLs)

- Kaggle Dataset [10, 11]: https://www.kaggle.com/datasets/akshayksingh/kidney-disease-dataset
- Shiny Web Application [15]: https://yueweixu.shinyapps.io/ckdPrediction
- GitHub R Package Repository [16]: https://github.com/YueweiXu-XJTLU/ckdPrediction

8. Reference

1. Kidney Disease: Improving Global Outcomes (KDIGO) CKD Work Group. (2024) KDIGO 2024 Clinical Practice Guideline for the Evaluation and Management of Chronic Kidney Disease. Kidney International, 105(Suppl 4S), S117–S314. (accessed 22 December 2025).
2. GBD Chronic Kidney Disease Collaboration. (2020) Global, regional, and national burden of chronic kidney disease, 1990–2017: a systematic analysis for the Global Burden of Disease Study 2017. Lancet, 395(10225), 709–733. doi:10.1016/S0140-6736(20)30045-3.
3. National Kidney Foundation. (n.d.) Global facts about kidney disease. (accessed 22 December 2025).
4. Lang,J., Scherzer,R., Grunfeld,C., Bacchetti,P., Choi,A.I. and Shlipak,M.G. (2018) Association of serum albumin levels with kidney function decline and incident chronic kidney disease in elders. Nephrology Dialysis Transplantation, 33(6), 986–992. doi:10.1093/ndt/gfx229.
5. Keith,D.S., Nichols,G.A., Gullion,C.M., Brown,J.B. and Smith,D.H. (2004) Longitudinal follow-up and outcomes among a population with chronic kidney disease in a large managed care organization. Archives of Internal Medicine, 164(6), 659–663. doi:10.1001/archinte.164.6.659.
6. Gaitonde,D.Y., Cook,D.L. and Rivera,I.M. (2017) Chronic Kidney Disease: Detection and Evaluation. American Family Physician, 96(12), 776–783.
7. Melamed,M.L., Bauer,C. and Hostetter,T.H. (2008) eGFR: Is It Ready for Early Identification of CKD? Clin. J. Am. Soc. Nephrol., 3(5), 1569–1572. doi:10.2215/CJN.02370508.
8. Alkahtani,F. and Alqarni,M. (2022) Machine Learning Approach for Chronic Kidney Disease Risk Prediction Combining Conventional Risk Factors and Novel Metabolic Indices. Applied Sciences, 12(23), 12001. doi:10.3390/app122312001.
9. Nguyen,D.P., Nguyen,T.T., Vu,T.T.L., Nguyen,N.N. and Nguyen,T.Q. (2025) Machine Learning Techniques in Chronic Kidney Diseases: A Comparative Study of Classification Model Performance. Bioinformatics and Biology Insights, 19, 11779322251356563. doi:10.1177/11779322251356563.
10. UCI Machine Learning Repository. (n.d.) Chronic Kidney Disease (Dataset 336). doi:10.24432/C5G020. (accessed 22 December 2025).
11. Kaggle. (n.d.) Kidney Disease Dataset (dataset by akshayksingh). (accessed 22 December 2025).
12. Breiman,L. (2001) Random Forests. Machine Learning, 45(1), 5–32. doi:10.1023/A:1010933404324.
13. Hoerl,A.E. and Kennard,R.W. (1970) Ridge Regression: Biased Estimation for Nonorthogonal Problems. Technometrics, 12(1), 55–67.
14. Cortes,C. and Vapnik,V. (1995) Support-vector networks. Machine Learning, 20, 273–297. doi:10.1023/A:1022627411411.
15. Xu,Y. (2025) ckdPrediction – Chronic Kidney Disease Prediction with Machine Learning. GitHub: YueweiXu-XJTLU/ckdPrediction. Available at: https://github.com/YueweiXu-XJTLU/ckdPrediction
16. Xu,Y. (2025) ckdPrediction – Chronic Kidney Disease Prediction with Machine Learning. Shiny web application (shinyapps.io). Available at: https://yueweixu.shinyapps.io/ckdPrediction